



Java Programming (24BCA404DS03) Important Short Question / Answer Wrt to Academic Year 2025-26 .

Compulsory Question with Answer

Q1) What is the use of listener interface in event handling?

Ans → **Use of Listener Interface in Event Handling :-**

- In Java, Listener Interface is used to handle events generated by user actions.
- It works with the Event Delegation Model of Java.
- A listener waits for events like mouse click, key press, or button action.
- When an event occurs, the corresponding method of the listener is executed.
- Listener interfaces are present in the `java.awt.event` package.
- Examples are `ActionListener`, `MouseListener`, and `KeyListener`.
- It helps in separating event handling code from GUI components.
- This makes programs modular and easier to manage.

Q2) What is synchronization in Java?

Ans → **Synchronization in Java :-**

- Synchronization is a process used to control access of multiple threads to shared resources.
- It prevents data inconsistency during multithreading.
- In Java, synchronization is achieved using the `synchronized` keyword.



- Only one thread can execute a synchronized block or method at a time.
- It avoids thread interference and race conditions.
- Synchronization improves data security in concurrent programs.
- It is mainly used in multithreaded applications.
- Example: updating a shared bank account balance safely.

Q3) Difference Between Byte Stream and Character Stream.

Ans → Following are difference between **Byte Stream** and **Character Stream** :-

- Byte Stream handles binary data like images, audio, and videos.
- Character Stream handles text data and characters.
- Byte Stream uses 8-bit bytes for input and output.
- Character Stream uses 16-bit Unicode characters.
- Byte Stream classes are **InputStream** and **OutputStream**.
- Character Stream classes are **Reader** and **Writer**.
- Byte Stream is faster for binary files.
- Character Stream is preferred for text files because it supports character encoding properly.

Q4) Tell any one method of the applet's life cycle.

Ans → **init() Method**

- The **init()** method is called when the applet starts for the first time.
- It is used for initialization of variables and objects.
- GUI components like buttons and text fields are also created here.
- This method executes only once in the applet life cycle.
- It prepares the applet before execution begins.
- The browser or applet viewer automatically calls this method.
- Syntax: **public void init()**
- It is considered the starting point of an applet program.



Q5) What is the use of finally block in exception handling?

Ans → **Use of Finally Block in Exception Handling :-**

- The **finally** block is used to execute important code after exception handling.
- It runs whether an exception occurs or not.
- It is mainly used for cleanup activities.
- Resources like files, database connections, and streams are closed here.
- The **finally** block comes after **try** and **catch** blocks.
- It improves program reliability and resource management.
- Even if an exception is not handled, **finally** usually executes.
- Syntax includes **try**, **catch**, and **finally** together in exception handling.

Q6) What is meant by user-defined package?

Ans →

- A user-defined package is a package created by the programmer in Java.
- It is used to organize related classes and interfaces.
- Packages help avoid naming conflicts between classes.
- They also provide access protection and code reusability.
- A package is created using the **package** keyword.
- Example: **package mypackage;**
- User-defined packages make programs modular and easier to maintain.
- Classes of the package can be imported using the **import** statement.

Q7) What is the purpose of the static import statement?

Ans →



- The static import statement allows direct access to static members of a class.
- It removes the need to use the class name repeatedly.
- Static methods and variables can be used directly in the program.
- It improves readability and reduces code length.
- Introduced in Java 5.
- Declared using the `import static` keyword.
- Example: `import static java.lang.Math.*;`
- Commonly used with mathematical functions like `sqrt()` and `pow()`.

Q8) Define string builder class.

Ans →

- `StringBuilder` is a class used to create mutable string objects in Java.
- It belongs to the `java.lang` package.
- Unlike `String`, its content can be modified without creating new objects.
- It is efficient for frequent string manipulation operations.
- Methods include `append()`, `insert()`, `delete()`, and `reverse()`.
- It improves memory usage and performance.
- `StringBuilder` is not synchronized, so it is faster than `StringBuffer`.
- Commonly used in loops and dynamic string operations.

Q9) What are the main features of Java?

Ans →

- Java is a high-level programming language.
- It is platform independent due to JVM.
- Java supports object-oriented programming concepts.



- It is secure and robust in nature.
- Java provides automatic garbage collection.
- It supports multithreading for concurrent execution.
- Java programs are portable and distributed.
- It has a rich API for application development.

Q10) What is JVM in Java?

Ans →

- JVM stands for Java Virtual Machine.
- It provides a runtime environment for Java programs.
- JVM converts bytecode into machine code.
- It makes Java platform independent.
- JVM manages memory allocation and garbage collection.
- It also handles exception management.
- JVM is a part of JRE.
- Without JVM, Java programs cannot execute.

Q11) What is Bytecode in Java?

Ans →

- Bytecode is the intermediate code generated by Java compiler.
- It is stored in `.class` files.
- Bytecode is executed by the JVM.
- It is machine-independent code.
- It helps Java achieve portability.
- The Java compiler converts source code into bytecode.
- JVM interprets or compiles bytecode into machine language.
- Bytecode improves program security and performance.

Q12) What is Garbage Collection in Java?

Ans →

- Garbage Collection is a memory management process in Java.



- It removes unused objects automatically.
- The JVM performs garbage collection.
- It helps free heap memory.
- It reduces memory leakage problems.
- Programmers do not need manual memory deallocation.
- The `System.gc()` method can request garbage collection.
- It improves application performance and efficiency.

Q13) What is Method Overloading?

Ans →

- Method overloading means defining multiple methods with the same name.
- The methods differ in parameters or data types.
- It supports compile-time polymorphism.
- Overloading improves code readability.
- Return type alone cannot overload methods.
- Constructors can also be overloaded.
- Java decides the method during compilation.
- Example: multiple `add()` methods with different arguments.

Q14) What is Inheritance in Java?

Ans →

- Inheritance allows one class to acquire properties of another class.
- The existing class is called parent or superclass.
- The new class is called child or subclass.
- It promotes code reusability.
- Java uses the `extends` keyword for inheritance.
- It supports hierarchical classification.
- Inheritance helps achieve runtime polymorphism.
- Example: Dog class inheriting Animal class properties.

Q15) Why is Java called a platform independent language?



Ans →

- Java programs run on different operating systems.
- Source code is converted into bytecode.
- Bytecode runs on JVM instead of hardware directly.
- Every operating system has its own JVM.
- Therefore, the same program works everywhere.
- It follows the principle “Write Once, Run Anywhere.”
- No recompilation is required for different platforms.
- This increases portability and flexibility of Java applications.

Q16) Why are constructors important in Java programs?

Ans →

- Constructors initialize objects during creation.
- They have the same name as the class.
- Constructors do not return any value.
- They help assign default values to variables.
- Constructors improve object initialization process.
- Java provides default and parameterized constructors.
- Constructors can also be overloaded.
- They are automatically called when objects are created.

Q17) Why are interfaces used in Java?

Ans →

- Interfaces provide complete abstraction in Java.
- They contain abstract methods by default.
- Interfaces support multiple inheritance.
- They improve code flexibility and reusability.
- Classes implement interfaces using **implements** keyword.
- Interfaces define common behavior for classes.
- They are useful in large applications.
- Example: **Runnable** interface in multithreading.



Q18) Why are wrapper classes needed in Java?

Ans →

- Wrapper classes convert primitive data types into objects.
- They are present in `java.lang` package.
- Collections framework works only with objects.
- Wrapper classes help in object manipulation.
- Examples include `Integer`, `Float`, and `Character`.
- They support autoboxing and unboxing.
- Wrapper classes provide useful utility methods.
- They improve flexibility in Java programming.

Q19) Why are loops used in Java programming?

Ans →

- Loops execute statements repeatedly.
- They reduce code repetition.
- Java provides `for`, `while`, and `do-while` loops.
- Loops improve program efficiency.
- They are useful for array processing.
- Conditions control loop execution.
- Infinite loops occur when conditions never become false.
- Loops make programs shorter and manageable.

Q20) Why are abstract classes important in Java?

Ans →

- Abstract classes provide partial abstraction.
- They cannot be instantiated directly.
- Abstract methods must be implemented by subclasses.
- They help define common behavior.
- Abstract classes may contain normal methods also.
- They improve code organization.



- Java uses the **abstract** keyword.
- They are widely used in framework development.

Q21) Write a Java program to display “Hello Java”.

Ans →

```
class Q21 {  
    public static void main(String args[]) {  
        System.out.println("Hello Java");  
    }  
}
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
PS C:\Users\HP\OneDrive\Desktop\GitDemo> cd "c:\Users\HP\OneDrive\Desktop\GitDemo\Java\  
Hello Java  
PS C:\Users\HP\OneDrive\Desktop\GitDemo\Java> |
```

Q22) Write a Java program to find the sum of two numbers.

Ans →

```
class Q22 {  
    public static void main(String args[]) {  
        int a = 10, b = 20;  
        int sum = a + b;  
  
        System.out.println("Sum = " + sum);  
    }  
}
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\HP\OneDrive\Desktop\GitDemo> cd "c:\Users\HP\OneDrive\Desktop\GitDemo\Java\"
Sum = 30
○ PS C:\Users\HP\OneDrive\Desktop\GitDemo\Java>
```

Q23) Write a Java program to demonstrate method overloading.

Ans →

```
J Q23.java
Java > J Q23.java > Java > Q23
1  class Q23 {
2
3      int add(int a, int b) {
4          return a + b;
5      }
6
7      int add(int a, int b, int c) {
8          return a + b + c;
9      }
10
11      Run main | Debug main | Run | Debug
12      public static void main(String args[]) {
13
14          Q23 d = new Q23();
15
16          System.out.println(d.add(a: 10, b: 20));
17          System.out.println(d.add(a: 10, b: 20, c: 30));
18      }
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\HP\OneDrive\Desktop\GitDemo> cd "c:\Users\HP\OneDrive\Desktop\GitDemo\Java\"
30
60
○ PS C:\Users\HP\OneDrive\Desktop\GitDemo\Java>
```



Q24) What is an Array in Java?

Ans →

- An array is a collection of similar data types.
- Arrays store multiple values in a single variable.
- Java arrays are fixed in size.
- Array elements are stored in contiguous memory locations.
- Indexing starts from 0.
- Arrays improve data handling efficiency.
- They can store primitive and object data types.
- Example: `int arr[] = {10,20,30};`

Q25) What is StringBuffer in Java?

Ans →

- `StringBuffer` is a mutable string class in Java.
- It belongs to the `java.lang` package.
- It allows modification of string contents.
- `StringBuffer` is synchronized and thread-safe.
- It provides methods like `append()` and `insert()`.
- It improves memory efficiency during string operations.
- It is slower than `StringBuilder`.
- Used in multithreaded applications.

Q26) What is StringTokenizer Class?

Ans →

- `StringTokenizer` is used to break strings into tokens.
- It belongs to the `java.util` package.
- Tokens are separated using delimiters.
- Default delimiter is whitespace.
- It provides methods like `nextToken()`.



- It is useful for parsing strings.
- It works faster for simple tokenization tasks.
- Example: splitting words from a sentence.

Q27) What is Exception Handling in Java?

Ans →

- Exception handling manages runtime errors in Java.
- It prevents abnormal termination of programs.
- Java uses `try`, `catch`, `finally`, `throw`, and `throws`.
- Exceptions are objects in Java.
- It improves program reliability.
- Common exceptions include `ArithmeticException` and `NullPointerException`.
- `catch` block handles exceptions.
- Proper handling ensures smooth program execution.

Q28) What is Static Import in Java?

Ans →

- Static import allows direct access to static members.
- It removes the need to use class names repeatedly.
- Declared using `import static`.
- It improves code readability.
- Introduced in Java 5.
- Static methods and variables are imported directly.
- Example: `import static java.lang.Math.*;`
- Useful for mathematical operations.

Q29) What are Packages in Java?

Ans →

- Packages are collections of related classes and interfaces.



- They help organize Java programs.
- Packages avoid naming conflicts.
- They provide access protection.
- Java has predefined and user-defined packages.
- Packages improve code reusability.
- Created using the `package` keyword.
- Imported using the `import` statement.

Q30) Why are arrays useful in programming?

Ans →

- Arrays store multiple values of the same type.
- They reduce the need for many variables.
- Arrays make data processing easier.
- They help in sorting and searching operations.
- Memory allocation becomes organized.
- Arrays improve program efficiency.
- Loops can easily access array elements.
- Widely used in mathematical and data applications.

Q31) Why is StringBuilder faster than StringBuffer?

Ans →

- `StringBuilder` is not synchronized.
- Lack of synchronization improves speed.
- It is suitable for single-threaded applications.
- Both classes provide mutable strings.
- `StringBuilder` uses less overhead.
- It performs better in repeated string modifications.
- Introduced in Java 5.
- Commonly used for high-performance applications.

Q32) Why is exception handling important in Java?



Ans →

- It prevents sudden program termination.
- Errors can be managed properly.
- Programs become more reliable.
- Exception handling improves debugging.
- Users receive meaningful error messages.
- Critical code can execute safely.
- Resources can be closed properly using **finally**.
- It increases software quality and stability.

Q33) Why are user-defined packages created?

Ans →

- They organize related classes together.
- Large projects become easier to manage.
- Naming conflicts are avoided.
- Packages improve code modularity.
- Access protection is provided.
- Reusability of classes increases.
- Maintenance becomes simpler.
- They support teamwork in software development.

Q34) Why is String class immutable in Java?

Ans →

- Immutable strings cannot be modified after creation.
- It improves security in Java applications.
- Strings become thread-safe automatically.
- Memory optimization is possible through string pool.
- Immutable strings improve performance.
- Hashcode caching becomes reliable.
- It prevents accidental data modification.
- Widely used in networking and database applications.



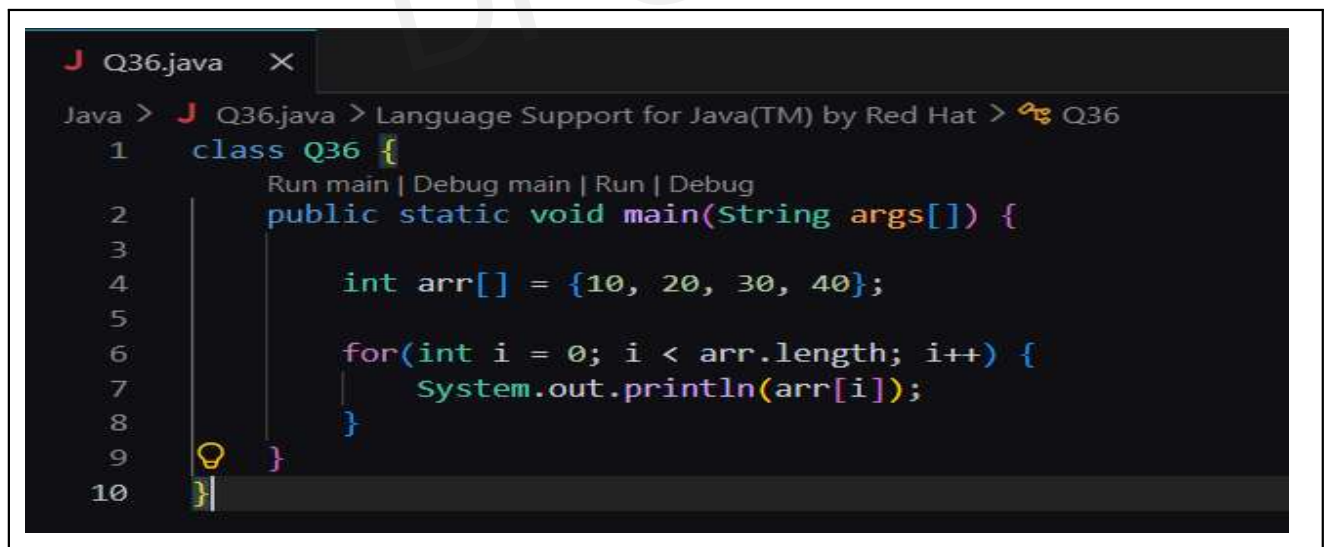
Q35) Why is the finally block used in Java?

Ans →

- **finally** executes important cleanup code.
- It runs whether exception occurs or not.
- Resources like files are closed safely.
- It prevents resource leakage problems.
- Used with **try** and **catch** blocks.
- It improves program reliability.
- Database connections can be closed here.
- Important cleanup tasks are guaranteed to execute.

Q36) Write a Java program to display array elements.

Ans →

A screenshot of an IDE window titled 'Q36.java'. The code defines a class 'Q36' with a 'main' method. Inside 'main', an integer array 'arr' is initialized with values {10, 20, 30, 40}. A 'for' loop iterates from 0 to arr.length-1, printing each element of the array using 'System.out.println(arr[i])'. The IDE interface includes a toolbar with 'Run main', 'Debug main', 'Run', and 'Debug' buttons.

```
1 class Q36 {  
2     public static void main(String args[]) {  
3  
4         int arr[] = {10, 20, 30, 40};  
5  
6         for(int i = 0; i < arr.length; i++) {  
7             System.out.println(arr[i]);  
8         }  
9     }  
10 }
```

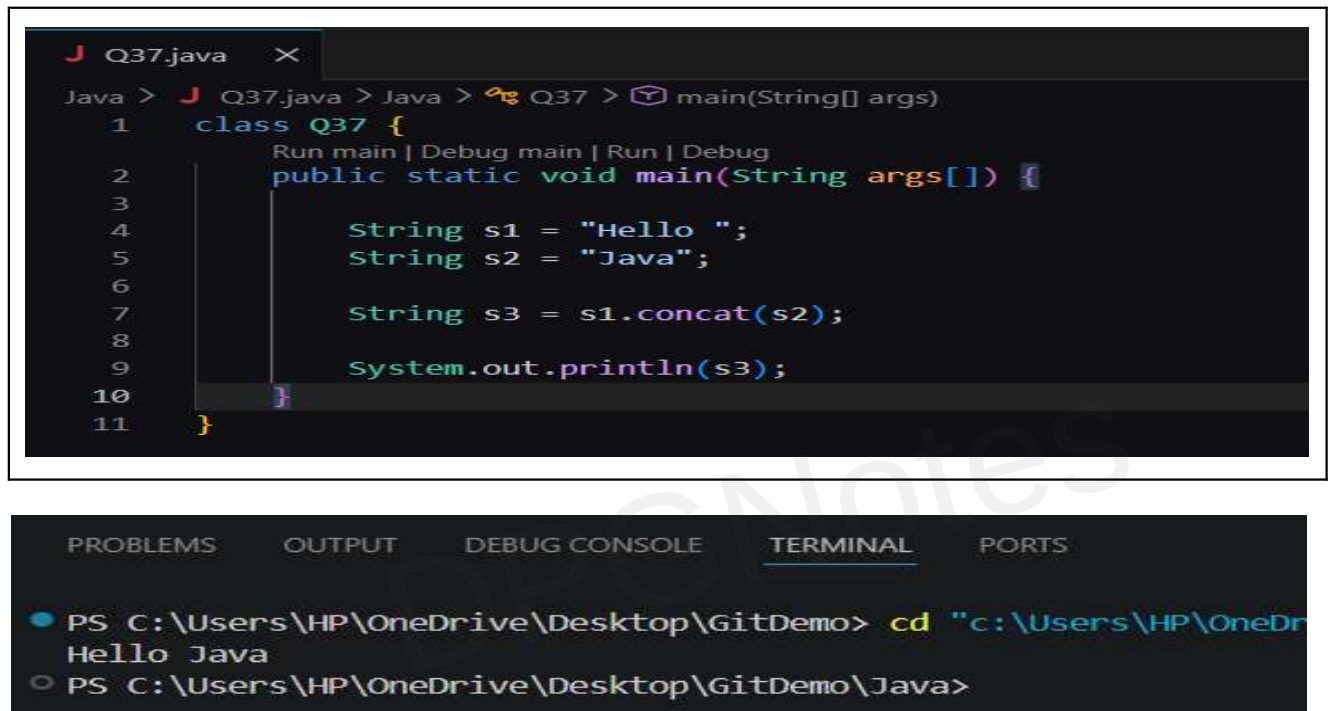
A screenshot of a terminal window with tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active, showing the command 'cd "c:\Users\HP\OneDrive\Desktop\GitDemo"' and the output of the Java program: '10', '20', '30', and '40' on separate lines. The prompt is 'PS C:\Users\HP\OneDrive\Desktop\GitDemo\Java>'.

```
PS C:\Users\HP\OneDrive\Desktop\GitDemo> cd "c:\Users\HP\OneDrive\Desktop\GitDemo"  
10  
20  
30  
40  
PS C:\Users\HP\OneDrive\Desktop\GitDemo\Java>
```



Q37) Write a Java program to concatenate two strings.

Ans →



The screenshot shows an IDE with a file named `Q37.java`. The code defines a class `Q37` with a `main` method. Inside the `main` method, two strings `s1` and `s2` are declared and assigned the values "Hello " and "Java" respectively. These strings are then concatenated using `s1.concat(s2)` and stored in `s3`. Finally, `s3` is printed using `System.out.println(s3)`. Below the code editor, the `TERMINAL` tab is active, showing the command `cd "c:\Users\HP\OneDrive\Desktop\GitDemo"` and the output `Hello Java`.

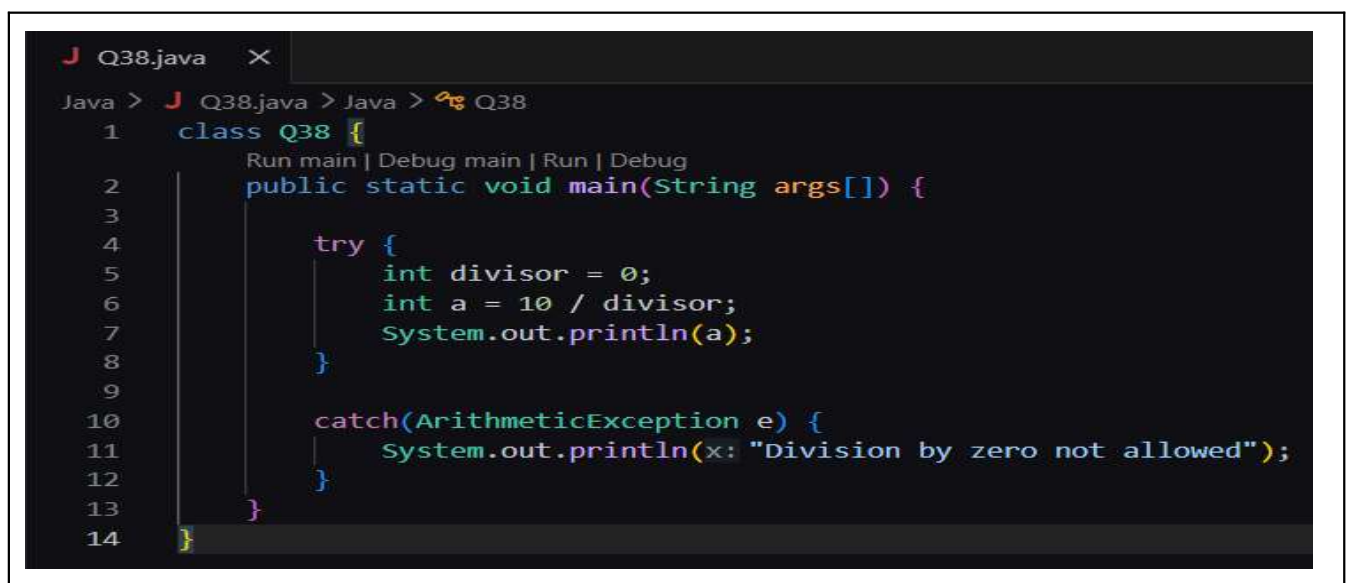
```
Java > J Q37.java > Java > Q37 > main(String[] args)
1  class Q37 {
    Run main | Debug main | Run | Debug
2      public static void main(String args[]) {
3
4          String s1 = "Hello ";
5          String s2 = "Java";
6
7          String s3 = s1.concat(s2);
8
9          System.out.println(s3);
10     }
11 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\HP\OneDrive\Desktop\GitDemo> cd "c:\Users\HP\OneDrive\Desktop\GitDemo"
Hello Java
○ PS C:\Users\HP\OneDrive\Desktop\GitDemo\Java>
```

Q38) Write a Java program using try-catch block.

Ans →



The screenshot shows an IDE with a file named `Q38.java`. The code defines a class `Q38` with a `main` method. Inside the `main` method, a `try` block is used to execute code that might throw an exception. In this case, it attempts to divide 10 by 0, which would cause an `ArithmeticException`. The `catch` block catches this exception and prints a message: `Division by zero not allowed`.

```
Java > J Q38.java > Java > Q38
1  class Q38 {
    Run main | Debug main | Run | Debug
2      public static void main(String args[]) {
3
4          try {
5              int divisor = 0;
6              int a = 10 / divisor;
7              System.out.println(a);
8          }
9
10         catch(ArithmeticException e) {
11             System.out.println(x: "Division by zero not allowed");
12         }
13     }
14 }
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\HP\OneDrive\Desktop\GitDemo> cd "c:\Users\HP\OneDrive\Desktop\GitDemo\Java"
Division by zero not allowed
○ PS C:\Users\HP\OneDrive\Desktop\GitDemo\Java>
```

Q39) What is Multithreading in Java?

Ans →

- Multithreading is the process of executing multiple threads simultaneously.
- A thread is a lightweight sub-process.
- It improves CPU utilization and performance.
- Java supports multithreading through **Thread** class and **Runnable** interface.
- Threads share the same memory area.
- It is useful in games and web servers.
- Multithreading enables concurrent task execution.
- It reduces program execution time.

Q40) What is Thread Life Cycle in Java?

Ans →

- A thread passes through different states during execution.
- States include New, Runnable, Running, Waiting, and Terminated.
- A thread starts in New state.
- **start()** method moves it to Runnable state.
- Scheduler selects it for Running state.
- Waiting state occurs during sleep or blocking.
- After completion, thread enters Terminated state.
- Life cycle controls thread execution flow.

Q41) What is Synchronization in Java?



Ans →

- Synchronization controls access to shared resources.
- It prevents data inconsistency in multithreading.
- Java uses **synchronized** keyword.
- Only one thread accesses synchronized code at a time.
- It avoids race conditions.
- Synchronization improves thread safety.
- Methods or blocks can be synchronized.
- Commonly used in banking and reservation systems.

Q42) What is Byte Stream in Java?

Ans →

- Byte Stream handles binary data in Java.
- It uses 8-bit bytes for input and output.
- Suitable for images, videos, and audio files.
- Main classes are **InputStream** and **OutputStream**.
- It supports file handling operations.
- Byte Stream works with raw data.
- It is faster for binary processing.
- Example: **FileInputStream** class.

Q43) What is Character Stream in Java?

Ans →

- Character Stream handles text data in Java.
- It uses Unicode characters.
- Main classes are **Reader** and **Writer**.
- Suitable for text files processing.
- It supports character encoding properly.
- Character Stream is slower than Byte Stream.
- It simplifies text handling operations.
- Example: **FileReader** and **FileWriter**.



Q44) What is Runnable Interface?

Ans →

- **Runnable** is an interface used for multithreading.
- It contains the **run()** method.
- A class implements Runnable to create threads.
- Thread object executes the **run()** method.
- It supports multiple inheritance indirectly.
- Better than extending Thread class in many cases.
- It improves code flexibility.
- Widely used in Java thread programming.

Q45) Why is multithreading preferred in modern applications?

Ans →

- Multiple tasks execute simultaneously.
- CPU resources are utilized efficiently.
- Applications become faster and responsive.
- Background tasks can run independently.
- It improves gaming and animation performance.
- Servers handle multiple users effectively.
- Time-consuming tasks run without freezing applications.
- It enhances overall user experience.

Q46) Why is synchronization necessary in Java?

Ans →

- Multiple threads may access shared data together.
- Simultaneous access can produce incorrect results.
- Synchronization ensures only one thread accesses data at a time.
- It prevents race conditions.
- Data consistency is maintained properly.
- Shared resources become thread-safe.



- It improves reliability in multithreaded applications.
- Common in banking and transaction systems.

Q47) Why are character streams preferred for text files?

Ans →

- Character streams support Unicode characters.
- Text is processed accurately with encoding support.
- They simplify reading and writing operations.
- Suitable for human-readable data.
- Byte streams may misinterpret text encoding.
- Character streams improve portability.
- Java provides Reader and Writer classes.
- Widely used for document processing applications.

Q48) Why is Runnable better than extending Thread class?

Ans →

- Java does not support multiple inheritance with classes.
- Runnable allows extending another class simultaneously.
- It separates thread logic from execution.
- Memory usage becomes efficient.
- Multiple threads can share the same Runnable object.
- It improves program flexibility.
- Runnable is widely recommended in Java.
- Better for large-scale applications.

Q49) Why are threads called lightweight processes?

Ans →

- Threads require less memory than processes.
- They share resources of the same process.
- Thread creation is faster.
- Context switching is quicker.



- Communication between threads is easy.
- They improve application performance.
- Threads reduce system overhead.
- Hence, they are called lightweight processes.

Q50) Why are input and output streams important?

Ans →

- Streams handle data transfer operations.
- They support file and console operations.
- Streams simplify reading and writing data.
- Java provides byte and character streams.
- They support efficient data management.
- Streams improve portability.
- Input streams read data while output streams write data.
- Essential for file handling applications.

Q51) Write a Java program to create a thread using Thread class.

Ans →

```
Q51.java
Java > Q51.java > Language Support for Java(TM) by Red Hat > Q51
1  class Q51 extends Thread {
2      @Override
3      public void run() {
4          System.out.println(x: "Thread is running");
5      }
6      Run main | Debug main | Run | Debug
7      public static void main(String args[]) {
8          Q51 t = new Q51();
9
10         t.start();
11     }
12 }
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\HP\OneDrive\Desktop\GitDemo> cd "c:\Users\HP\OneDrive\Desktop
Thread is running
○ PS C:\Users\HP\OneDrive\Desktop\GitDemo\Java>
```

Q52) Write a Java program using Runnable interface.

Ans →

```
J Q52.java
Java > J Q52.java > Java > Q52 > main(String[] args)
1 class Q52 implements Runnable {
2     @Override
3     public void run() {
4         System.out.println(x: "Runnable thread executed");
5     }
6     Run main | Debug main | Run | Debug
7     public static void main(String args[]) {
8         Q52 d = new Q52();
9         Thread t = new Thread(d);
10        t.start();
11    }
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\HP\OneDrive\Desktop\GitDemo> cd "c:\Users\HP\OneD
● Runnable thread executed
○ PS C:\Users\HP\OneDrive\Desktop\GitDemo\Java>
```

Q53) Write a Java program to read data from keyboard.

Ans →



```
J Q53.java
Java > J Q53.java > Java > Q53 > main(String[] args)
1  import java.util.Scanner;
2  class Q53 {
    Run main | Debug main | Run | Debug
3      public static void main(String args[]) {
4          try (Scanner sc = new Scanner(System.in)) {
5              System.out.println(x: "Enter your name:");
6              String name = sc.nextLine();
7              System.out.println("Hello " + name);
8          }
9      }
10 }
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● PS C:\Users\HP\OneDrive\Desktop\GitDemo> cd "c:\Users\HP\OneD
Enter your name:
Akshat
Hello Akshat
○ PS C:\Users\HP\OneDrive\Desktop\GitDemo\Java> █
```

Q54) What is an Applet in Java?

Ans →

- An applet is a small Java program that runs inside a web browser.
- It is mainly used for internet-based applications.
- Applets extend the `Applet` class.
- They do not use the `main()` method.
- Applets run under JVM support.
- They provide graphical user interfaces.
- Applets are controlled by browsers or applet viewers.
- They follow a specific applet life cycle.

Q55) What is Applet Life Cycle?

Ans →



- Applet life cycle defines stages of applet execution.
- Important methods are `init()`, `start()`, `stop()`, and `destroy()`.
- `init()` initializes the applet.
- `start()` begins execution.
- `stop()` pauses execution temporarily.
- `destroy()` removes the applet from memory.
- Browser controls these methods automatically.
- It manages proper applet execution flow.

Q56) What is Event Handling in Java?

Ans →

- Event handling manages user-generated actions in Java.
- Events include mouse clicks and keyboard actions.
- Java follows Delegation Event Model.
- Listener interfaces handle events.
- Event source generates the event object.
- Event handlers process the events.
- It is widely used in GUI applications.
- Example: button click handling.

Q57) What is Delegation Event Model?

Ans →

- Delegation Event Model is used for event handling in Java.
- It separates event generation from handling.
- Event source generates events.
- Listener objects receive and handle events.
- It improves flexibility and efficiency.
- Multiple listeners can handle one event.
- Present in `java.awt.event` package.
- Common in GUI programming applications.



Q58) What is GUI Programming in Java?

Ans →

- GUI stands for Graphical User Interface.
- It provides visual interaction with users.
- Java GUI uses AWT and Swing components.
- GUI applications contain buttons and text fields.
- Frames act as main windows.
- Layout managers arrange components properly.
- GUI improves user experience.
- Event handling is essential in GUI programming.

Q59) What are Layout Managers in Java?

Ans →

- Layout managers control arrangement of GUI components.
- They organize components automatically.
- Java provides different layout managers.
- Common layouts are FlowLayout and BorderLayout.
- GridLayout arranges components in rows and columns.
- Layout managers improve GUI appearance.
- They make interfaces platform independent.
- Used in AWT and Swing applications.

Q60) Why are applets different from Java applications?

Ans →

- Applets run inside browsers while applications run independently.
- Applets do not contain `main()` method.
- Java applications start from `main()` method.
- Applets follow applet life cycle methods.
- Browser controls applet execution.
- Applications can access system resources fully.



- Applets have restricted security permissions.
- Applets mainly support web-based environments.

Q61) Why is event handling important in GUI applications?

Ans →

- GUI applications interact with users continuously.
- Event handling responds to user actions.
- It improves application interactivity.
- Buttons and menus depend on events.
- Keyboard and mouse operations generate events.
- Listener interfaces process these actions.
- Event handling improves user experience.
- Essential for responsive applications.

Q62) Why are layout managers used in Java GUI?

Ans →

- Components need proper arrangement in windows.
- Layout managers automate component positioning.
- They reduce manual alignment work.
- GUI becomes platform independent.
- Window resizing is managed automatically.
- Components appear organized and attractive.
- Different layouts suit different interfaces.
- They simplify GUI development process.

Q63) Why is the `init()` method important in applets?

Ans →

- `init()` executes when applet starts first time.
- It initializes variables and GUI components.
- Memory allocation is often done here.
- Initialization occurs only once.



- It prepares the applet for execution.
- Browser automatically calls `init()`.
- Important setup operations are performed here.
- Considered the starting stage of applet life cycle.

Q64) Why are listener interfaces used in Java?

Ans →

- Listener interfaces handle generated events.
- They respond to user interactions.
- Event handling becomes modular and organized.
- Java provides many predefined listeners.
- Examples include `ActionListener` and `MouseListener`.
- Listener methods execute when events occur.
- They support GUI programming efficiently.
- Essential part of Delegation Event Model.

Q65) Why are frames important in GUI programming?

Ans →

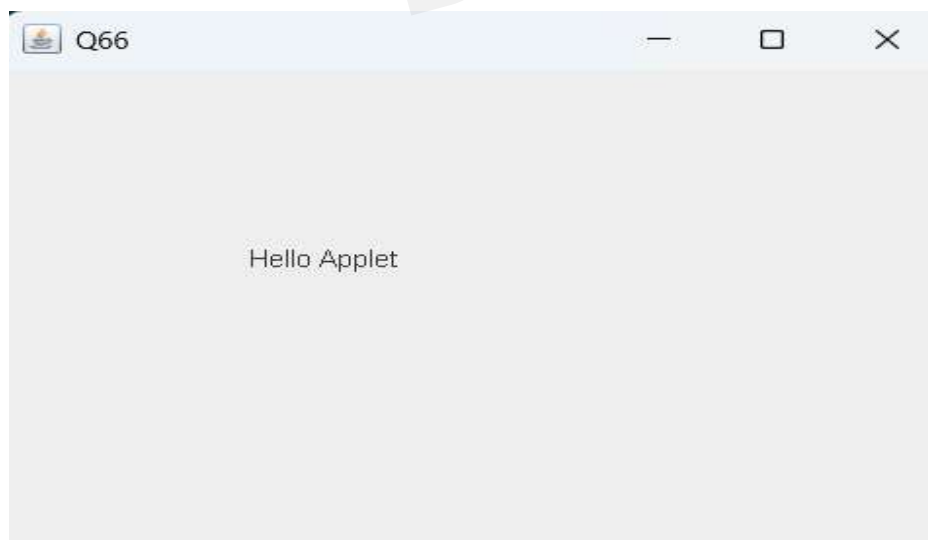
- Frames act as main windows in GUI applications.
- GUI components are added inside frames.
- Frames provide user interaction space.
- They support menus and controls.
- Java AWT provides `Frame` class.
- Frames improve graphical presentation.
- Applications become user-friendly.
- Essential for desktop GUI applications.

Q66) Write a Java applet program to display a message.

Ans →



```
Q66.java
Java > J Q66.java > ...
1  import javax.swing.JFrame;
2  import javax.swing.JPanel;
3  import java.awt.Graphics;
4  public class Q66 extends JPanel {
    Run main | Debug main | Run | Debug
5      public static void main(String[] args) {
6          JFrame f = new JFrame(title: "Q66");
7          f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
8          f.setSize(width: 400, height: 300);
9          f.add(new Q66());
10         f.setLocationRelativeTo(c: null);
11         f.setVisible(b: true);
12     }
13     @Override
14     protected void paintComponent(Graphics g) {
15         super.paintComponent(g);
16         g.drawString(str: "Hello Applet", x: 100, y: 100);
17     }
18 }
```



Q67) Write a Java program to create a Frame window.

Ans →



```
J Q67.java •
Java > J Q67.java > Java > Q67 > main(String[] args)
1  import java.awt.*;
2  class Q67 {
    Run main | Debug main | Run | Debug
3      public static void main(String args[]) {
4          Frame f = new Frame(title: "Java Frame");
5          f.setSize(width: 300, height: 300);
6          f.setVisible(b: true);
7      }
8  }
```



Q68) What is Encapsulation?

Ans →

- Encapsulation is the process of binding data and methods into a single unit.
- It is one of the important principles of OOP.
- In Java, classes are used for encapsulation.
- Data members are usually declared private.
- Methods provide controlled access to data.
- Getter and setter methods are commonly used.
- It improves data security and modularity.



- Example: wrapping variables and methods inside a class.

Q69) What is Inheritance?

Ans →

- Inheritance allows one class to acquire properties of another class.
- It supports code reusability in Java.
- Parent class is called superclass.
- Child class is called subclass.
- Java uses **extends** keyword for inheritance.
- It supports method overriding.
- Inheritance helps achieve runtime polymorphism.
- Example: Dog class inheriting Animal class properties.

Q70) What is JVM?

Ans →

- JVM stands for Java Virtual Machine.
- It provides runtime environment for Java programs.
- JVM converts bytecode into machine code.
- It makes Java platform independent.
- JVM manages memory allocation.
- It also handles garbage collection.
- JVM is a part of JRE.
- Without JVM, Java programs cannot execute properly.

Q71) What is Final Keyword?

Ans →

- **final** is a keyword used to restrict modification.
- Final variable value cannot be changed.
- Final methods cannot be overridden.
- Final classes cannot be inherited.
- It improves security and reliability.



- Used for constants in Java.
- Syntax: `final int x = 10;`
- Commonly used in object-oriented programming.

Q72) What do you mean by Extends Keyword?

Ans →

- `extends` keyword is used for inheritance in Java.
- It allows one class to inherit another class.
- Child class acquires parent class properties.
- It promotes code reusability.
- Syntax: `class B extends A`
- Subclass can access protected and public members.
- It supports method overriding.
- Used to establish IS-A relationship between classes.

Q73) What are Modifiers?

Ans →

- Modifiers define properties and behavior of classes and members.
- Java provides access and non-access modifiers.
- Access modifiers control visibility.
- Examples are `public`, `private`, and `protected`.
- Non-access modifiers include `static`, `final`, and `abstract`.
- Modifiers improve security and control.
- They help manage inheritance and accessibility.
- Used with classes, methods, and variables.

Q74) What is CLASSPATH?

Ans →

- CLASSPATH is an environment variable in Java.
- It specifies the location of class files and packages.



- JVM uses CLASSPATH to find required classes.
- It helps in loading external libraries.
- Incorrect CLASSPATH causes class loading errors.
- It can include directories and JAR files.
- Set using command line or environment settings.
- Important for Java application execution.

Q75) What do you understand by Polymorphism?

Ans →

- Polymorphism means “many forms” in Java.
- It allows one method or object to perform different tasks.
- It is an important feature of Object-Oriented Programming.
- Java supports compile-time and runtime polymorphism.
- Method overloading is compile-time polymorphism.
- Method overriding is runtime polymorphism.
- Polymorphism improves flexibility and reusability of code.
- Example: one method name performing different operations with different parameters.

Q76) What is an Instance?

Ans →

- An instance is an object created from a class.
- It represents a real entity in memory.
- Each instance has its own data and behavior.
- Objects are created using the **new** keyword.
- Instance variables belong to objects, not classes.
- Multiple instances can be created from one class.
- Each instance occupies separate memory space.
- Example: **Student s = new Student();** creates an instance of **Student** class.

